DS-AG-032: LLMs and LangChain Framework Assignment Answers

Question 1: What are Large Language Models (LLMs) and how do they function?

Answer:

Large Language Models (LLMs) are deep learning models trained on massive text datasets using the Transformer architecture. They learn statistical relationships between words and generate human-like text by predicting the next token in a sequence. Training consists of pretraining on large corpora followed by fine-tuning or instruction tuning for specific tasks. LLMs are widely used for text generation, summarization, translation, question answering, coding assistance, and conversational AI.

Question 2: Discuss the impact of LLMs on traditional software development approaches.

Answer:

LLMs accelerate software development by generating code, documentation, test cases, and debugging suggestions. They reduce repetitive work, improve developer productivity, enable natural language programming, and simplify knowledge retrieval. However, developers must validate generated code because LLMs can hallucinate or produce insecure solutions.

Question 3: What are the key advantages and limitations of using LLMs in real-world applications?

Answer:

Advantages: High productivity, multilingual support, automation, natural interaction, scalability, and rapid content generation.
Limitations: Hallucinations, bias, privacy concerns, high computational cost, outdated knowledge, and lack of true reasoning.

Question 4: Describe how different industries are being transformed by the use of LLMs. Provide examples.

Answer:

Healthcare: Clinical documentation and medical assistants.
Education: Personalized tutoring and content generation.
Finance: Report generation and fraud analysis.
Legal: Contract summarization and legal research.
Customer Support: AI chatbots and ticket automation.
Software: Code generation using GitHub Copilot and ChatGPT.

Question 5: Compare and contrast LangChain and LlamaIndex. What unique problems does each solve?

Answer:

LangChain is an orchestration framework for building LLM applications with prompts, chains, memory, agents, and tools.
LlamaIndex focuses on indexing, organizing, and retrieving information from documents for Retrieval-Augmented Generation (RAG).

LangChain manages workflows, whereas LlamaIndex specializes in document retrieval.

Question 6

```
Answer:

Python Code:
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate

llm = ChatOpenAI(model="gpt-4o-mini")
prompt = ChatPromptTemplate.from_template("Answer: {question}")
chain = prompt | llm
print(chain.invoke({"question":"What is LangChain?"}))

Sample Output:
LangChain is a framework for building applications powered by LLMs.
```

Question 7

```
Answer:

Python Code:
from langchain.tools import Tool
import requests

def get_weather(city):
    return requests.get(f"https://wttr.in/{city}?format=3").text

# Pass weather output to LLM

Sample Output:
Delhi: 35°C, Sunny
The weather in Delhi is sunny with a temperature of 35°C.
```

Question 8

```
Answer:

Python Code:
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader

docs = SimpleDirectoryReader("data").load_data()
index = VectorStoreIndex.from_documents(docs)
query = index.as_query_engine()
print(query.query("Summarize the document"))

Sample Output:
The document discusses artificial intelligence and its applications.
```

Question 9

```
Answer:
Python Code:
```

```
Use LlamaIndex for document retrieval and LangChain for prompt orchestration.

retrieved_context = index.as_query_engine().query(question)
Pass retrieved_context to LangChain LLM.

Sample Output:
Q: What is RAG?
A: Retrieval-Augmented Generation combines retrieval with LLM reasoning.
```

Question 10

```
Answer:

Proposed Solution:
• Store legal documents in a vector database.
• Use LlamaIndex to create searchable indexes.
• Use LangChain to orchestrate prompts, retrieval, memory, and responses.
• Use GPT-4 or similar LLM for summarization and question answering.

Benefits:
- Fast legal document search
- Accurate summaries
- Natural language querying
- Improved lawyer productivity

Sample Code:
from langchain.chains import RetrievalQA
# Build RetrievalQA chain with indexed legal documents.
```